An Agent-Based Limit Order Book for Stealth-Execution Research

Technical documentation of the `order_book` package: simulator, validation and the measurement experiments

Manu

October 23, 2026

Abstract

This document describes the simulator used in the study *Hiding in the Order Flow: Learning to Execute Against Liquidity Providers that Detect Metaorders*. It is a deterministic, single-instrument, price–time matching engine surrounded by a heterogeneous population of agents (zero-intelligence noise flow, fundamentalists and chartists with Brock–Hommes switching, institutional traders executing labelled metaorders in five styles, Avellaneda–Stoikov market makers with configurable detection responses and volatility fragility, and an order-anticipation trader) in which detection of large orders emerges from agents that see only public data. We document every rule, equation and default parameter, the randomness model that makes runs exactly reproducible and counterfactuals paired, the validation methodology against stylised facts and the square-root impact law, the measured results of the validation, the automated tests, and the commands that reproduce everything. Part II documents the measurement phase that needs no reinforcement learning: the execution task, the baseline strategies and their calibration, the held-out and learned detectors, detector scoring and the statistical protocol, and the detectability (E2) and leakage (E3) experiments. Sections are written so that they can be lifted into the paper's methods section and appendices.

Contents

I The simulator **3**

1 Purpose and scope **3**

2 Design rules and how the code enforces them **3**

3 Architecture **3**

4 Matching engine (`order_book.book`) **4**

5 Accounting and routing (`order_book.market`) **5**

6 Public state (`order_book.public_state`) **5**

7 Randomness and reproducibility **6**

8 Agents **7**

8.1 Hidden value (`FundamentalValue`) 7

8.2 Noise flow (`NoiseFlow`) 7

8.3 Speculators (`Speculators`) 7

8.4 Institutional traders (`InstitutionalTrader`) . . . 8
8.5 Market makers (`MarketMaker`) . . . 9
8.6 Order anticipator (`OrderAnticipator`) . . . 10

9 Detectors (`order_book.detectors`) 10

10 Configuration 11

11 Simulator loop, snapshots and external participants 13

12 Validation 13
12.1 Metrics . . . 13
12.2 Impact measurement . . . 13
12.3 Targets and status . . . 14

II Measurement without reinforcement learning 15

13 The execution task and the executor agent (`order_book.execution.task`) 15

14 Baseline strategies (`order_book.execution.strategies`) 16

15 Calibration (`order_book.execution.calibration`) 17

16 Held-out and learned detectors 17

17 Scoring and statistical protocol (`order_book.scoring`) 18

18 Experiments 18
18.1 Adversary levels and observers . . . 18
18.2 E2: detectability of classic strategies . . . 19
18.3 E3: leakage cost . . . 19

19 Reproduction of Part II 19

III Verification and status 20

20 Tests 20

21 Reproduction 21

22 Status against the research plan and next steps 21

Part I
The simulator

1 Purpose and scope

The research question is how much of an execution's cost is caused by *information leakage* (other agents detecting the order and reacting) as opposed to mechanical market impact, and whether a learning executor can reduce that leakage against detectors it has never seen. Answering it needs a market in which (i) the reaction to a large order is produced by other agents rather than assumed by an impact model, (ii) detection is both possible and genuinely hard because other institutions trade large orders too, (iii) every agent, detector and executor sees only public information, and (iv) runs are exactly reproducible so that the cost of leakage can be measured by counterfactual simulation with common random numbers.

The `order_book` package implements that market. This document covers the simulator and its validation (Phases 0 and 1 of the research plan). The execution task, the reinforcement-learning executor and the adversary ladder beyond levels A0, A1 and A4 are later phases and are only mentioned where the simulator already provides for them.

Units. Prices are integer *ticks* (the default initial mid is 10 000 ticks, i.e. \$100.00 at a one-cent tick), quantities are integer *lots*, cash is in ticks×lots, and time is in *steps*. One step is a configurable unit of wall-clock time (`SimConfig.step_seconds`, default one second); every rate in the configuration is per step.

2 Design rules and how the code enforces them

The research plan imposes ten rules on the implementation. Table 1 lists them with the mechanism and, where one exists, the automated test that would fail if the rule were broken.

3 Architecture

```
order_book/
  book.py            OrderBook: the matching engine; knows nothing about agents
  market.py          Account, Market: accounting and order routing
  public_state.py    PublicState: the per-step public view everyone may read
  config.py          frozen dataclasses for every parameter; SimConfig aggregates
    them
  detectors/         Detector interface; FlowDetector (rule-based, level A1)
  agents/            NoiseFlow, Speculators, InstitutionalTrader, MarketMaker,
                     OrderAnticipator, FundamentalValue, MetaorderLog
  simulator.py       Simulator: one seeded run; streams, snapshots, external agents
  presets.py         adversary levels A0/A1/A4, private valuations, no institutions
  history.py         per-step recording of an analysis run (plots, animation)
  validation.py      stylised facts, impact statistics, pass/fail table
  impact.py          paired impact measurement with injected metaorders
  execution/         ExecutionTask, ExecutorAgent, Strategy; the baseline strategies;
                     Calibration and its fitting (Part II)
  detectors/         + features.py, statistical.py (A2), learned.py (A3)
  scoring.py         ROC AUC, detection delay, false alarms; bootstrap, CVaR, Holm
  experiments.py     AdversarySpec, EpisodeSpec, E2Config, E3Config, runner,
    summaries
  env.py             Gymnasium wrapper (prototype; replaced in Phase 3)
```

Rule	Mechanism	Test
Determinism	one root `SeedSequence(seed)`; one child stream per agent in fixed order (§7)	`test_foundations`: identical event log from (config, seed), from a snapshot, and after `reseed`
No hidden information	agents hold a `Market` handle but only call its order methods and read `Market.public`, their own account and their own orders; trade reports with identities never leave settlement; metaorder labels live in a log agents only append to	`test_foundations`: a guard that raises if any agent, while acting, reads another account, another's orders, the tape, the book tables, the hidden value's history or the metaorder labels; mutation-tested
Book independence	`OrderBook` imports nothing from the package; agents never touch it except through `Market`	import graph (§3)
Configuration over code	every parameter is a field of a frozen dataclass in `config.py`; experiments differ only by `dataclasses.replace`	`test_simulator`
Invariants	never crossed at rest; cash and shares conserved; no negative free balances; reservations equal what rests	`test_foundations`: asserted after every one of 300 steps
Performance	`scripts/benchmark.py` reports steps/s and a profile	reported in §12.3
Two logging modes	`analysis=True` keeps tape, trade log, hidden value and labels; `analysis=False` keeps rolling aggregates only	`test_simulator`
Report, do not tune silently	`scripts/validate.py` prints measured values against the plan's targets; metric definitions are fixed in `validation.py`	§12.3
Snapshots	`Simulator.snapshot()/restore()`	`test_foundations`
Small steps	this document records what was changed, measured and proposed	—

Table 1: The implementing rules of the plan and their enforcement.

```
scripts/              CLI entry points: run, batch, benchmark, animate, burn_in,
    validate,
                      impact, calibrate, train_detector, e2_detectability, e3_leakage
tests/                pytest suite
docs/                 this document
```

Dependencies point strictly downward: `book` ← `market`, `public_state` ← `detectors` ← `agents` ← `simulator` ← `presets`, `history`, `validation`, `impact`, `execution`, `env` ← `experiments` ← `scripts`. The book never imports anything above it; detectors read only `PublicState`; agents reach the book only through `Market`.

4 Matching engine (`order_book.book`)

`OrderBook` is a deterministic single-instrument matching engine. It generates no liquidity and assumes nothing about who trades or why; the caller serialises actions, and call order determines arrival priority.

- **Priority.** Best price first, first-in-first-out within a price. Trades execute at the resting order's price.
- **Order types.** A *limit* order matches at its limit or better and rests for the remainder. A *market* order consumes available liquidity and cancels any remainder (immediate-or-cancel). `Market.ioc` is a limit that cancels what would rest.

- **Self-trade prevention.** Default `cancel_newest`: the incoming remainder is cancelled when it would match the same client's resting order. `cancel_oldest` removes the resting order and continues; `cancel_both` removes both; `none` permits self-matching. Fills made before the encounter stand.
- **Modification.** Reducing size retains priority. Increasing size or changing price replaces the order with a new identifier and priority. Invalid modifications leave the original intact.
- **Ownership.** Cancellation and modification check the client id when it is given.
- **Bounded window.** Each side admits prices within N ticks (`num_levels`, default 20) of an anchor: its own best price when populated, one tick inside the opposite best when it is empty, and the last trade price when both sides are empty. Orders outside the window are rejected; an improvement of the best price can *evict* resting orders that fall outside the new window, and the report lists them so that the accountant can release reservations. Empty sides never manufacture quotes: `mid_price` and `spread` are undefined unless both sides rest. This bounded book is not a normal exchange rule; it keeps observations finite and is part of the environment's dynamics.
- **Reports.** Every operation returns a report naming the maker order of every fill, the orders removed by self-trade prevention and by eviction, and requested/filled/resting/cancelled quantities that always sum. Reports are shared read-only objects.
- **Complexity.** Best quotes, mid, spread, per-level volume, per-client order lookup and single order lookup are $O(1)$.

5 Accounting and routing (`order_book.market`)

`Market` owns every `Account` (cash, shares, reserved cash, reserved shares) and is the only path from an agent to the book. Resting bids reserve price $\times$ quantity of cash and resting asks reserve shares; reservations are running totals updated on placement, fill (both sides), cancellation, self-trade cancellation and eviction, so that free balances are $O(1)$ and

$$\text{free cash} = \text{cash} - \text{reserved cash} \geq 0, \qquad \text{free shares} = \text{shares} - \text{reserved shares} \geq 0$$

hold at all times. A limit order is capped at what the account can fund; a market buy walks the ask levels and takes only what the free cash can pay for. Orders cut to zero are counted in `budget_rejects`; orders outside the window in `window_rejects`. Every fill moves price$\times$quantity of cash from buyer to seller and the shares the other way, so

$$\sum_i \text{cash}_i \text{ and } \sum_i \text{shares}_i \text{ are conserved exactly}$$

across the population (`Simulator.check_conservation`). Nobody can short or borrow.

In analysis mode the market keeps a *tape* of (step, price, quantity, aggressor side, buyer, seller); the buyer and seller identities are for offline analysis only and no agent reads the tape. The market also hands the public state an *anonymous* list of the step's aggressive fills, (side, price, quantity), and the aggressive volume that enters the signed flow window. `Market.hide(cid)` removes one client's aggressive trades from both (its trades still happen, settle, move the price and count in the volume totals): this deliberately breaks the no-identity rule and exists only for the analysis-mode counterfactual that separates impact cost from leakage cost (Part II, §18.3).

6 Public state (`order_book.public_state`)

`PublicState` is refreshed once per step from the book and the step's trades. It is the only market information any participant may read. With m_t the mid (the last trade price when a side is

empty), $\Delta m_t = m_t - m_{t-1}$, V_t^b, V_t^s the step's buyer- and seller-initiated volume and $V_t = V_t^b + V_t^s$, and EWMA weights $\alpha_f = 2/(\tau_f + 1)$, $\alpha_s = 2/(\tau_s + 1)$ with spans $\tau_f = 10$ and $\tau_s = 100$:

$$\bar{m}_t^f = \bar{m}_{t-1}^f + \alpha_f(m_t - \bar{m}_{t-1}^f), \qquad \bar{m}_t^s = \bar{m}_{t-1}^s + \alpha_s(m_t - \bar{m}_{t-1}^s), \tag{1}$$

$$\sigma_t^{2,f} = \sigma_{t-1}^{2,f} + \alpha_f(\Delta m_t^2 - \sigma_{t-1}^{2,f}), \qquad \sigma_t^{2,s} = \sigma_{t-1}^{2,s} + \alpha_s(\Delta m_t^2 - \sigma_{t-1}^{2,s}), \tag{2}$$

$$\bar{V}_t^f = \bar{V}_{t-1}^f + \alpha_f(V_t - \bar{V}_{t-1}^f), \qquad \bar{V}_t^s = \bar{V}_{t-1}^s + \alpha_s(V_t - \bar{V}_{t-1}^s). \tag{3}$$

Momentum indicators for three lookback pairs $(\ell_f, \ell_s) \in \{(5, 30), (10, 60), (20, 120)\}$ are differences of EMAs of the mid with those spans, $\mu_t^{(k)} = \mathrm{EMA}_{\ell_f}(m)_t - \mathrm{EMA}_{\ell_s}(m)_t$. Signed aggressive flow over a rolling window of $W = 20$ steps gives the imbalance

$$I_t = \frac{\sum_{u=t-W+1}^{t} V_u^b - \sum_{u=t-W+1}^{t} V_u^s}{\sum_{u=t-W+1}^{t} V_u} \in [-1, 1], \tag{4}$$

maintained with a ring buffer rather than a tape scan. Finally an *attention* multiplier, used by the noise flow to cluster activity, is

$$a_t = \min\Big(3,\ \max\Big(\tfrac{1}{2},\ 1 + \tfrac{1}{2}\Big(\sqrt{\sigma_t^{2,f}/\sigma_t^{2,s}} - 1\Big) + \tfrac{1}{2}\Big(\bar{V}_t^f/\bar{V}_t^s - 1\Big)\Big)\Big), \tag{5}$$

held at 1 until the slow statistics have seen a full span (`warmed_up`). The public state also exposes best bid and ask, spread, last trade, and the step's volume. It contains no interpretation of these statistics: anything that infers a metaorder from them is a *detector* (§9).

7 Randomness and reproducibility

A run is fully determined by a `SimConfig` and an integer seed. The simulator creates one root `numpy.random.SeedSequence(seed)` and spawns from it, in a fixed order, one independent child generator (PCG64) for

1. the *scheduler*, which draws the random permutation in which background agents act each step;
2. the *hidden value* process;
3. each background agent in construction order: market makers, the noise pool, institutional traders, the speculator population (one stream for the whole array-backed population), order anticipators;
4. any *external* participant added later through the simulator's `new_stream()` or `add_external()` methods.

Consequences:

- **Exact replay.** The same (config, seed) reproduces the event log byte for byte; so does restoring a snapshot and continuing. `reseed(seed')` re-derives every existing stream from a new root in place, so all holders follow it.
- **Independence of population changes.** Removing the institutions or adding an anticipator (which is constructed last) leaves every earlier agent's stream untouched.
- **Paired counterfactuals (common random numbers).** An external agent acts *before* the background population and outside the scheduler's permutation, with its own stream. Running the same seed with and without it therefore shares the scheduler's and the hidden value's draws exactly, and every background agent's draws *up to state dependence*: an agent that draws a state-dependent number of variates (the noise pool draws Poisson(λa_t) orders; speculators draw per active member) diverges only after the market itself has diverged because of the injected order. The difference between the two mid paths is then the injected order's impact and the market's reaction to it with the market's own randomness removed. This is the instrument used for the square-root impact measurement (§12.2) and, in Phase 2, for the leakage decomposition $\mathrm{IS}_{\text{visible}} - \mathrm{IS}_{\text{invisible}}$.

- **Scope.** A Gymnasium episode reseeds a restored snapshot with a seed drawn from the environment's own generator, so parallel environments are independent and reproducible from the environment seed.

8 Agents

Every step, the hidden value advances, external agents act in fixed order, background agents act in the scheduler's random order, the public state is refreshed, and the detectors are updated. Agents read `Market.public`, the book's best quotes and levels, their own account and their own orders. Cost scales with events, not with the number of notional traders.

8.1 Hidden value (`FundamentalValue`)

Exogenous information arrival is a random walk in ticks with optional reversion to its initial value,

$$v_t = v_{t-1} + \rho\,(v_0 - v_{t-1}) + \sigma_v\,\varepsilon_t, \qquad \varepsilon_t \sim N(0,1), \tag{6}$$

with $\sigma_v = 0.3$ and $\rho = 0$ by default. Only speculators see it, and only through private noise; it drives beliefs, not prices. This is standard in agent-based models, and the paper's claim is limited accordingly: microstructure and impact emerge from agents, the long-run price level does not.

Private valuations switch. With `FundamentalConfig.private = True` there is no shared value: each speculator i holds a private valuation that drifts with its own noise and adapts toward the traded mid,

$$v_{i,t} = v_{i,t-1} + \kappa_a\,(m_{t-1} - v_{i,t-1}) + \sigma_v\,\varepsilon_{i,t}, \tag{7}$$

updated for all members once per step ($\kappa_a = 0.01$ by default). The shared path is still advanced (so the stream structure is unchanged) but unused. The plan asks for the main results under both information models.

8.2 Noise flow (`NoiseFlow`)

One pooled account submits zero-intelligence order flow. Each step it first cancels a random subset of its resting orders, Binomial$(n_{\text{resting}}, p_c)$ of them with $p_c = 0.015$, then submits $N_t \sim$ Poisson$(\lambda\,a_t)$ new orders ($\lambda = 2.5$, a_t the attention multiplier). Each order has size $U\{1,\dots,q_{\max}\}$ ($q_{\max} = 12$) and is a buy with probability

$$p_{\text{bid}} = \tfrac{1}{2} + w\,(c_t - \tfrac{1}{2}), \qquad c_t = \frac{\text{cash}}{\text{cash} + \text{shares}\cdot m_t}, \tag{8}$$

($w = 0.5$), i.e. tilted toward the pool's cash share of wealth so it cannot exhaust its cash or its stock in a long trend. A fraction $f_m = 0.15$ of orders are market orders at half size (takers are smaller than makers), a fraction $f_x = 0.05$ are marketable limits at the opposite best, and the rest rest $1 +$ Geometric ticks behind the opposite best with mean offset $\bar{d} + 1$ ($\bar{d} = 3$).

8.3 Speculators (`Speculators`)

An array-backed population of $n = 40$ members, each either a *fundamentalist* (mode 0) or a *chartist* (mode 1). Each step a vectorised draw selects the active members (each with probability $p_a = 0.05$); only they do any work. An active member first cancels any resting order it has, then:

Fundamentalist. Forms the belief $b_{i,t} = v_t + \beta_i + \eta_{i,t}$ with a permanent private bias $\beta_i \sim N(0, 3^2)$ and fresh observation noise $\eta_{i,t} \sim N(0, 1^2)$ (ticks). With gap $g = b_{i,t} - m_t$ and threshold $\theta = 2$ ticks it does nothing if $|g| < \theta$; otherwise it trades toward its belief with size

$$q = \min\left(q_{\max},\ \max(1, \mathrm{round}(|g| - \theta))\right), \qquad q_{\max} = 20, \tag{9}$$

capped by its position limit (the share endowment by default). For $|g| \geq g_a = 8$ ticks it uses a market order; otherwise a passive limit at $\lfloor b - \theta \rfloor$ (buy) or $\lceil b + \theta \rceil$ (sell), kept at least one tick inside the opposite best.

Chartist. Follows the public momentum indicator $\mu_t^{(k_i)}$ of its assigned lookback pair. If it has a position and the signal has reversed (position $\cdot\, \mu < 0$) or faded ($|\mu| < \theta_c/4$), it exits by market order in chunks of at most $q_c = 10$ lots. Otherwise, if $|\mu| \geq \theta_c$ it enters in the direction of μ with size $\min(q_c, \max(1, \mathrm{round}(s\,(|\mu| - \theta_c))))$, $s = 2$ lots per tick, within the position limit. The threshold $\theta_c = 6$ ticks is the main lever on return autocorrelation (§12.3).

Brock–Hommes switching. Every $T_s = 50$ steps, each mode's fitness F_k is updated with the mean realised wealth return (in percent) of its current members over the block, $F_k \leftarrow F_k + \lambda_F(\bar{r}_k - F_k)$, $\lambda_F = 0.1$, and every member re-draws its mode with

$$P(\text{chartist}) = \frac{1}{1 + e^{-\beta_s(F_1 - F_0)}}, \qquad \beta_s = 2. \tag{10}$$

8.4 Institutional traders (`InstitutionalTrader`)

Institutional traders (four by default) are what makes detection hard: their metaorders look like an executor's and create false positives for every detector. Each trader runs at most one metaorder at a time.

Arrival. While idle, a metaorder starts with probability $\lambda_I = 5 \times 10^{-4}$ per step (a discrete Poisson process; expected idle time 2 000 steps).

Side. With inventory relative to the initial endowment $x = (\text{shares} - \text{shares}_0)/\text{shares}_0$, the metaorder is a buy with probability

$$p_{\text{bid}} = \tfrac{1}{2} - \tfrac{1}{2}\tanh(\pi_I\, x), \qquad \pi_I = 2, \tag{11}$$

so each trader's holding mean-reverts and the aggregate institutional flow stays balanced over long runs. (Without this, the population's inventory random-walks and, over tens of thousands of steps, exhausts the other agents' budgets; see §12.3.)

Size and participation. Size is log-normal, $Q = \min(Q_{\max}, \mathrm{round}(Q_{\text{med}}\, e^{\sigma_Q Z}))$ with $Q_{\text{med}} = 600$ lots, $\sigma_Q = 0.7$, $Q_{\max} = 3\,000$, capped by what the account can fund. The target participation rate is $\pi \sim U(0.02, 0.20)$ of market volume. The implied duration uses the public slow volume EWMA $\bar{V}^s$ (a fallback of 5 lots/step before warm-up),

$$T = \mathrm{clamp}\Big(\frac{Q}{\pi\, \bar{V}^s},\ 100,\ 4\,000\Big) \text{ steps}. \tag{12}$$

With a market volume of about 5 lots/step, Q ranges roughly from 150 to 2 500 lots, which overlaps the executor's Q of 5–20% of 2 000 steps of volume (500–2 000 lots).

Execution styles. Each metaorder draws one of five styles (uniformly by default; `style_weights` can change this). Children are sent every $\Delta = 5$ steps. Progress is read from the trader's own account (shares relative to the start), so passive fills need no callbacks. Let R be the remaining quantity, S the number of slices left, X the quantity executed so far.

`twap_market` every Δ steps, a market order of $\lceil R/S \rceil$.

`twap_random` as above with the child size multiplied by $U(1-j, 1+j)$ and the next interval by $U(1-j, 1+j)$, $j = 0.5$ (the "naive" way to hide; Yang & Zhu predict randomisation).

`twap_passive` each slice of $\lceil R/S \rceil$ is posted as a limit order at the trader's own best (best bid for a buy); at the slice deadline the resting order is cancelled and the unfilled part of the slice is crossed by market order ("catch-up"). This is what practitioners run.

`pov` every Δ steps, bring X up to $\min(Q, \lfloor \pi \cdot (\text{public volume since the start}) \rfloor)$ by market order, where the trader itself accumulates the public per-step volume; over the final tenth of the horizon it switches to catch-up TWAP.

`front_loaded` an Almgren–Chriss-like trajectory, $\text{remaining}(t) = Q \sinh(k(T-t))/\sinh(kT)$ with $kT = 2$, brought up to schedule by market order every Δ steps.

Whatever remains at the planned end is forced through by market order, as an execution desk would do, and its cost counts in full.

Labels. In analysis mode every metaorder is recorded in `Simulator.metaorders.records` as a `MetaorderRecord`: id, client, side, style, intended size, participation, start, planned end, arrival mid, actual end, executed quantity and notional, from which the average price and the implementation shortfall in basis points,

$$\text{IS} = \text{sign} \cdot \frac{\bar{p} - m_{\text{arrival}}}{m_{\text{arrival}}} \times 10^4, \qquad \text{sign} = +1 \text{ for a buy}, -1 \text{ for a sell}, \tag{13}$$

follow. These are the ground truth for scoring detectors (ROC AUC, detection delay, false alarms). Agents only *open* their own record; the guard test fails if any agent reads the list. In training mode no labels are kept.

Injection. `InstitutionalTrader.begin(side, size, participation, style, duration)` starts a metaorder on demand. Together with `Simulator.add_external` this is how the impact experiment injects a single metaorder into an otherwise identical market.

8.5 Market makers (`MarketMaker`)

Four two-sided post-only quoters in the spirit of Avellaneda–Stoikov. With inventory I relative to the initial holding, $\gamma = 0.003$, $\kappa = 1$, horizon $H = 50$ steps and $\sigma^2 = \max(\sigma_t^{2,f}, \sigma^2_{\min})$ ($\sigma^2_{\min} = 0.25$ ticks^2/step):

$$r_t = m_t - \text{clamp}(I\,\gamma\sigma^2 H,\ -\bar{s},\ \bar{s}), \qquad h_t = \text{clamp}\Big(\tfrac{1}{2}(\gamma\sigma^2 H + \tfrac{2}{\gamma}\ln(1+\tfrac{\gamma}{\kappa})),\ 1,\ \bar{s}\Big), \tag{14}$$

with $\bar{s} = 10$ ticks capping both the inventory skew and the half-spread so that a volatility spike cannot make the quoter cross the book. Quotes are bid $= r_t - h_t - \omega^b$, ask $= \max(r_t + h_t + \omega^a,\ \text{bid} + 2)$, then clamped to be post-only against everyone else's best (`best_excluding` the maker's own order). Quote size on each side is drawn uniformly from $[5, 20]$ lots when a quote is (re)placed. A side is cancelled and replaced only when its target price has moved at least one tick from the resting quote or its remaining size has fallen below half of target, so the maker keeps queue position otherwise. The side that would push $|I|$ beyond $I_{\max} = 200$ is pulled.

Detection response. A maker whose configuration names a detector reads its signal $s_t \in [-1, 1]$ (positive: buy metaorder suspected). The pressured side is the ask for $s_t > 0$ and the bid for

$s_t < 0$. The response is

$$r_t \mathrel{+}= \delta_r \, s_t, \qquad \omega^{\text{pressured}} = \delta_w \, |s_t|, \qquad \text{size}^{\text{pressured}} \mathrel{\times}= \max(0,\ 1 - \delta_q \, |s_t|), \tag{15}$$

with $\delta_r = 4$ ticks, $\delta_w = 3$ ticks and $\delta_q = 0.7$. The mapping from signal to action is the same for every adversary level; only the detector changes, so differences in execution cost across levels come from detection quality alone. Running makers with and without detection side by side makes false alarms costly: a maker that widens on a false alarm loses volume and spread income to the others.

Fragility. With the volatility ratio $\varrho_t = \sqrt{\sigma_t^{2,f} / \sigma_t^{2,s}}$, both quote sizes are scaled by $\max(0,\ 1 - \phi \, \max(\varrho_t - 1, 0))$, $\phi = 0.5$, and when $\varrho_t > \varrho_{\text{pull}} = 3$ both quotes are cancelled and stay away for a cooldown of 20 steps. Liquidity therefore thins exactly when a large order makes the price move fast, which is how market impact becomes concave and how fast moves amplify.

8.6 Order anticipator (`OrderAnticipator`)

The order-anticipation trader (adversary level A4; none by default) reads a detector and trades on it with market orders. With $|s_t| \geq \theta_A = 0.5$ it trades in the signal's direction $\min\left(c,\ \text{round}(\varphi \, (|s_t| - \theta_A)/(1 - \theta_A))\right)$ lots per step ($\varphi = 2$ lots/step, chunk $c = 10$) within a position limit of $P_{\max} = 200$ lots; with $|s_t| < \theta_{\text{fade}} = 0.2$ it unwinds c lots per step; in between it holds. With `contrarian_first` it trades *against* the detected flow (supplying liquidity, as van Kervel & Menkveld report for the start of institutional executions) until the signal has been confident for `switch_after` $= 50$ steps, then with it. A false alarm costs the anticipator the round trip, so detection is not free for it either.

9 Detectors (`order_book.detectors`)

A detector implements `update(public_state)`, called once per step after the public state is refreshed, and exposes a signed `signal` $\in [-1, 1]$ and probabilities `p_buy`, `p_sell` $\in [0, 1]$ that a metaorder is active on each side. Detectors read only `PublicState`. The simulator keeps a registry `Simulator.detectors` by name; market makers and anticipators whose configuration names a detector hold a reference to it. No detector ever receives a flag saying an executor is active.

Flow detector (level A1). The rule-based detector keeps an EWMA of the public flow imbalance with span $\tau_d = 10$,

$$M_t = M_{t-1} + \tfrac{2}{\tau_d + 1} \, (I_t - M_{t-1}), \tag{16}$$

and emits

$$s_t = \text{sign}(M_t) \cdot \min\Big(1,\ \frac{|M_t| - \theta_d}{1 - \theta_d}\Big)^+, \qquad p_t^{\text{buy}} = s_t^+, \quad p_t^{\text{sell}} = (-s_t)^+, \tag{17}$$

with threshold $\theta_d \in \{0.2, 0.3, 0.45\}$ for the *lenient*, *medium* and *strict* variants (`FLOW_THRESHOLDS` in `presets`). Held-out statistical detectors (A2), a learned classifier (A3) and best-response detectors (A5) plug in behind the same interface in later phases.

Adversary presets. `presets.adversary(level, threshold)` returns the configuration for A0 (no maker reacts to any detector, no anticipator), A1 (two of four makers react to the flow detector) and A4 (A1 plus one anticipator). Only the detector and the presence of the anticipator differ.

10 Configuration

All parameters are fields of frozen dataclasses in `order_book/config.py`, aggregated in `SimConfig`. Frozen dataclasses are hashable, picklable and compared by value; variants are made with `dataclasses.replace`:

```
from dataclasses import replace
from order_book import SimConfig, Simulator
from order_book.config import PopulationConfig, market_makers

cfg = replace(SimConfig(),
              population=replace(PopulationConfig(), institutions=8,
                                 market_makers=market_makers(n=6, detecting=3)))
sim = Simulator(seed=0, config=cfg, analysis=True)
```

Table 2: Configuration fields and defaults.

Field	Default	Meaning
BookConfig		
num_levels	20	bounded-window depth per side (ticks)
mid	10 000	initial mid in ticks
distance_from_mid	1	initial anchor offset before any trade
PublicStateConfig		
lookbacks	(5,30), (10,60), (20,120)	momentum EMA span pairs
fast, slow	10, 100	EWMA spans of the fast and slow statistics
flow_window	20	steps of signed aggressive volume in the imbalance window
FlowDetectorConfig		
memory	10	EWMA span of the imbalance
threshold	0.3	no signal below this $\lvert M_t \rvert$ (0.2 lenient, 0.45 strict)
FundamentalConfig		
sigma	0.3	ticks/step of the value's random walk
reversion	0	pull toward the initial value per step
private	False	private adaptive valuations instead of one shared value
adapt	0.01	private mode: pull of each valuation toward the mid
NoiseFlowConfig		
rate	2.5	Poisson orders per step before attention
cancel_prob	0.015	per resting order per step
max_qty	12	order size uniform on 1..max_qty
mean_offset	3	mean ticks behind the opposite best for passive limits
market_frac, cross_frac	0.15, 0.05	share of market orders (half size), of marketable limits
balance_weight	0.5	tilt of the bid probability toward the cash share
SpeculatorConfig		
p_act	0.05	probability each member acts per step
bias_sd, obs_noise	3, 1	permanent bias and fresh noise on the value (ticks)
threshold, aggressive_gap	2, 8	fundamentalist: minimum gap; gap for a market order
max_qty, max_position	20, endowment	fundamentalist size cap; absolute position cap

Field	Default	Meaning
chart_threshold	6	chartist: minimum $\|\mu\|$ to enter (3 gives trending returns, §12.3)
chart_max_qty, size_per_tick	10, 2	chartist size cap; lots per tick of excess signal
switch_every, beta, fitness_memory	50, 2, 0.1	Brock–Hommes block, intensity of choice, fitness EWMA
InstitutionalConfig		
start_rate	5×10^{-4}	probability per idle step of starting a metaorder
size_median, size_sigma, size_max	600, 0.7, 3 000	log-normal size (lots) and cap
participation	(0.02, 0.20)	uniform target share of market volume
duration	(100, 4 000)	clamp on the implied duration (steps)
interval	5	steps between child orders / slice length
styles, style_weights	all five, uniform	execution styles and their probabilities
jitter	0.5	randomised TWAP relative jitter
front_load	2	kT of the sinh schedule
volume_fallback	5	lots/step assumed before the public EWMA warms up
inventory_pull	2	tanh scale of the side tilt toward the endowment
MarketMakerConfig	(one per maker; `market_makers(n, detecting)` builds a tuple)	
gamma, kappa, horizon	0.003, 1, 50	Avellaneda–Stoikov parameters
min_var, max_skew	0.25, 10	variance floor; cap on skew and half-spread (ticks)
size, refill_frac	(5,20), 0.5	quote size range; requote when below this share
max_inventory	200	pull the side that would exceed this
detector	None / "flow"	detector to react to
detect_skew, detect_widen, detect_size_cut	4, 3, 0.7	response per unit signal
fragility, pull_ratio, cooldown	0.5, 3, 20	size scale slope; pull threshold; steps pulled
OrderAnticipatorConfig		
detector	"flow"	detector to trade on
threshold, fade	0.5, 0.2	enter above; unwind below
rate, chunk	2, 10	lots/step at full signal; max lots per order
max_position	200	absolute position cap
contrarian_first, switch_after	False, 50	trade against the flow first; steps before switching
PopulationConfig		
market_makers	4 (2 detecting)	tuple of MarketMakerConfig
institutions, speculators, anticipators	4, 40, 0	counts
EndowmentConfig		(shares; cash = shares × mid)
maker, noise, institution, speculator, anticipator	300, 6 000, 5 000, 400, 1 000	initial holdings
SimConfig		
step_seconds	1	wall-clock meaning of one step

Client ids are assigned in construction order: makers $0..n_m - 1$, the noise pool, institutional traders, speculators, anticipators, then external participants. `Simulator.groups` maps display names (`MM1`, `Noise`, `Inst1`, `Speculators`, `Antic1`, `Ext1`, ...) to client ids.

11 Simulator loop, snapshots and external participants

`Simulator(seed, config, analysis)` builds the book, the market, the hidden value, the detector registry, the metaorder log and the population. `step()` performs one step as described in §8; `run(n)` repeats it. `snapshot()` pickles the whole simulator and `restore(blob)` returns an independent copy that continues identically, so training episodes can start from a bank of post-burn-in states instead of burning in every episode. `join(cash, shares)` opens an account for an externally driven trader (the RL executor acts between `run` calls); `add_external(cls, cash, shares, **kwargs)` adds an agent that acts every step before the background population with its own stream (§7). `check_conservation()` asserts the cash, share and free-balance invariants.

12 Validation

12.1 Metrics

`order_book.validation` runs several seeds in analysis mode after a burn-in and computes, from the mid path m_t, the spread, the tape and the metaorder labels:

Liquidity median spread in ticks; share of steps with an empty side.

Returns autocorrelation of k-step returns $m_{t+k} - m_t$ ($k = 10$) at lags $1, 2, 5, 10, 20, 50$ (lag 5 therefore spans 50 steps); the target applies from lag 5 on.

Fat tails kurtosis $\mathbb{E}[r^4]/\mathbb{E}[r^2]^2$ of returns at horizons $1, 5, 20, 100$ steps.

Clustering autocorrelation of $|r|$ ($k = 10$) at lags $1, 5, 20, 50, 100, 200$.

Flow memory autocorrelation of trade signs ($+1$ buyer-initiated) at lags $1, 10, 100, 1\,000$ trades.

Budgets budget rejects per trade and their balance across sides.

Speed background steps per second in analysis mode.

12.2 Impact measurement

The square-root law, peak impact $\propto (Q/V)^\delta$ with $\delta \approx 0.5$, is measured in two ways, both normalising size by $V = \bar{V} \cdot T_{\text{ref}}$, the mean market volume per step times a reference horizon of $T_{\text{ref}} = 2\,000$ steps (the executor's horizon), so that Q/V is on the same axis as the execution task and spans the size distribution. (Normalising by the volume over the metaorder's own life pins Q/V to about $\pi/(1+\pi)$ and makes the exponent unidentifiable.)

Natural metaorders. For every labelled institutional metaorder the signed mid change from the arrival mid is read at completion ("impact at completion"), at its maximum over the execution ("peak"), and 500 steps after completion ("residual"). Sizes are binned by Q/V into log-spaced quantile bins, impact is averaged per bin (negatives included), and log(impact) is regressed on $\log(Q/V)$ over bins with a positive mean. Reversion is reported as pooled ratios, $\sum \text{residual} / \sum \text{peak}$ and $\sum \text{end} / \sum \text{peak}$. These statistics are *noisy*: the metaorder's impact of a few ticks sits on a mid path whose standard deviation over the metaorder's life is of the same order or larger, so the per-metaorder standard deviation of the impact is 10–30 ticks and hundreds of metaorders per bin are needed. They are reported as informational.

Paired injection (primary). For each seed and each fraction $f \in \{0.01, 0.02, 0.05, 0.1, 0.2\}$, an external institutional trader is added (§7) and, after the burn-in, injects one buy metaorder of $Q = f \cdot \bar{V} \cdot T_{\text{ref}}$ lots executed as TWAP market orders over exactly T_{ref} steps (participation f). The impact path is the difference between this run's mid and the mid of the identical seed without the injected trader. Impact at completion, peak and residual are read off the paired path, binned by f and fitted as above. The pairing removes the market's own randomness up to state dependence and typically reduces the per-run standard deviation to a few ticks. This

is the measurement behind the "Square-root impact (paired)" row of the validation table and `scripts/impact.py`.

12.3 Targets and status

Table 3 lists the plan's starting targets and the values measured at the time of writing with `python -m scripts.validate -seeds 5 -steps 30000` (seeds 0–4, 2 000 steps of burn-in, the default configuration). The numbers are reported, not enforced; where a target fails, the measured value and the proposed change are given.

Check	Target	Measured (5 seeds × 30k steps, final defaults)
Book integrity	never crossed; exact conservation	asserted every step in tests: pass
Liquidity	median spread 1–3 ticks; empty side $< 0.1\%$ of steps	spread 1.00; empty side 0.000%: pass
Return autocorrelation	$\lvert ac\rvert < 0.05$ from lag 5	max over seeds and lags ≥ 5: 0.044: pass
Fat tails	kurtosis $\gg 3$ at short horizons, falling toward 3	12.7 at 1 step $\rightarrow$ 6.9 at 100 steps: pass
Volatility clustering	$ac(\lvert r\rvert) > 0$, slowly decaying	0.20 (lag 1), 0.15 (lag 20), 0.09 (lag 100): pass
Order-flow memory	trade-sign $ac > 0$, slowly decaying	0.46 (lag 1), 0.09 (lag 10), 0.04 (lag 100): pass
Square-root impact	$\delta \in [0.4, 0.7]$	paired: 0.69 at completion, 0.64 at peak ($n = 25$); mean impact for $Q/V = 0.01/0.02/0.05/0.1/0.2$: 0.8/1.2/2.7/3.9/6.0 ticks; natural: 0.60 at completion, 0.54 at peak ($n = 181$): pass
Impact decay	partial reversion after the end	paired: impact at completion 52% of peak, 42% of peak remains 500 steps later; natural: 32% and 20%: pass
Budgets	$< 1\%$ of trades rejected, balanced	0 rejects in $\approx$ 190k trades: pass
Speed	$\geq 10\,000$ steps/s	14 800 steps/s in analysis mode during validation; 15 200 in the benchmark (training mode, 20k steps, one core): pass

Table 3: Validation targets and measured values with the final defaults (`python -m scripts.validate -seeds 5 -steps 30000`, seeds 0–4, 2 000 steps of burn-in).

What the validation found and changed. Two defects of the market were found only on runs longer than 10 000 steps and fixed before these numbers were taken:

1. **Budget exhaustion.** With a random side choice, the institutional population's aggregate inventory random-walked; on one seed it net-sold 7 500 lots over 30 000 steps, the noise pool ran out of cash, the speculators sat at their position caps and the mid fell to 1 954 against a hidden value of 9 963 (kurtosis 120, 637 rejected buys). The inventory-reverting side choice (§8.4) and larger endowments for the absorbing agents (noise pool 6 000 shares, speculators 400) removed it: all seeds now track the hidden value and rejects are negligible.
2. **Measurement power.** The natural-metaorder impact statistics cannot resolve the exponent with a few hundred metaorders; the paired injection method was introduced for this reason.

Return autocorrelation sweep. The one failing stylised fact is positive autocorrelation of 10-step returns: $ac(1) \approx 0.15$–0.3 and, on some seeds, $|ac| > 0.05$ at lags 5–10 (50–100 steps). Ablations show that it comes from slow price adjustment: without speculators the mid trends strongly ($ac \approx 0.25$–0.3 at all lags) because the makers quote around a mid they themselves set, so inventory skew feeds back on itself and only the fundamentalists anchor the price; chartists amplify it. A sweep over the fundamental volatility $\sigma_v \in \{0.3, 0.6\}$, the chartist threshold $\theta_c \in \{3, 6\}$ and the maker risk aversion $\gamma \in \{0.002, 0.006\}$ (5 seeds $\times$ 20 000 steps each) gave, for $\max|ac|$ over lags ≥ 5: raising σ_v makes trending worse (0.19–0.53); $\theta_c = 6$ with $\gamma = 0.002$ gives 0.053 (mean 0.023), $\gamma = 0.006$ with $\theta_c = 3$ gives 0.095, and both together 0.096; kurtosis and liquidity stay on target throughout. A second sweep on held-out seeds 5–9 over $\theta_c \in \{6, 8\}$ and $\gamma \in \{0.002, 0.003\}$ gave $\max|ac|$ of 0.445 (one trending seed; mean 0.044), 0.053 (mean 0.016), 0.070 and 0.078 respectively. The defaults were set to $\theta_c = 6$, $\gamma = 0.003$, and the reference run of Table 3 (seeds 0–4, which were not used to choose between these two values) passes every target. The remaining positive autocorrelation at lags 1–2 (10–20 steps; $ac(1) \approx 0.17$) is allowed by the target and reflects the gradual price adjustment to metaorders that the study is about.

Alternative presets. The same validation was run on 5 seeds $\times$ 10 000 steps for the alternative configurations (these are shorter runs and the impact exponent on $n = 25$ paired runs is correspondingly noisier):

- **Private valuations** (`-preset private`, $\kappa_a = 0.01$): liquidity, fat tails (kurtosis 103 $\rightarrow$ 5) and order-flow memory pass, but impact is nearly permanent (90% of the peak at completion, 95% after 500 steps) with $\delta \approx 0.9$, because every valuation adapts toward the new price and nothing anchors it. Slower adaptation ($\kappa_a = 10^{-3}$, 2×10^{-4}) restores reversion but then speculators drift to their position caps and budgets fail. The private information model is implemented and runs, but it needs its own calibration (likely a larger speculator endowment and a reversion term) before the main results are reported under it; this is an open item.
- **A4** (one anticipator): with the original anticipator defaults (4 lots/step, limit 300) the anticipator was about 40% of market volume when active and created a self-reinforcing loop (its own trades raised the imbalance it reacted to), trending the market ($\max|ac| = 0.54$) and making impact superlinear ($\delta = 1.24$ at completion). With the adopted defaults (2 lots/step, limit 200) the market stays regular ($\max|ac| = 0.08$, kurtosis 15 $\rightarrow$ 5) while impact is still amplified ($\delta = 0.79$ at completion, 0.66 at peak; 26% of peak remains after 500 steps), which is the effect the adversary ladder is meant to produce. A4 is not required to pass the background-market validation (E1 is run on the background market only), but it must not destabilise it.

Part II
Measurement without reinforcement learning

This part implements Phase 2 of the plan: everything needed for experiments E2 and E3 and for the Phase 2 gate (TWAP's leakage cost significantly above zero at one size or more). Nothing here learns by reinforcement; the executor runs fixed strategies, and the only trained component is the A3 classifier, which never sees an executor.

13 The execution task and the executor agent (`order_book.execution.task`)

Task. An `ExecutionTask` fixes the side (buy or sell), the size as a fraction f of expected market volume over the horizon, $Q = \text{round}(f\,\bar{V}\,T)$ with $f \in \{0.05, 0.10, 0.20\}$, the horizon

$T = 2\,000$ steps, the decision interval $N = 5$ steps, and the hard limits: at most two resting orders, no marketable limit more than 5 ticks through the opposite best, and an optional cap on the market-order size per decision as a share of expected interval volume (unused by the baselines, so that *Immediate* can exist; the RL environment will set it). $\bar{V}$ is the mean traded volume per step over the burn-in, read from the public tape, so it is identical for every strategy and adversary level on the same seed.

Executor. `ExecutorAgent` is an external participant (§7): it acts first in every step with its own stream and outside the scheduler's permutation. It starts at a given step (the end of the burn-in), records the arrival mid, and every N steps hands its strategy an `ExecState` and applies the returned `Decision` (optionally cancel resting orders; a market order; then a limit order at a price, default own best). Fills are read off the executor's own account (shares and cash relative to the start), so passive fills need no callbacks, and each fill is tagged aggressive or passive. At T the agent cancels what rests and sends the remainder by market order, repeating for up to 50 steps if the book is thin; the forced quantity is reported and its cost counts in full. The result records executed and forced quantity, notional, aggressive share, orders sent and the fill list, and

$$\text{IS} = \text{sign} \cdot \frac{\bar{p} - m_{\text{arrival}}}{m_{\text{arrival}}} \times 10^4 \text{ bps}, \qquad \text{completion} = \frac{Q_{\text{executed}} - Q_{\text{forced}}}{Q}. \tag{18}$$

What a strategy sees. `ExecState`: elapsed and remaining time, decisions left, Q, executed and remaining quantity, aggressive quantity, arrival mid, mid, best quotes, spread, the public flow imbalance, public volume since the start, the executor's own resting orders, the expected volume per step, the executor's own share of the aggressive volume in the public flow window (its *footprint*), and the `PublicState` itself. It never sees any detector's output or any other agent's state.

14 Baseline strategies (`order_book.execution.strategies`)

Let R be the remaining quantity, S the decisions left, X the executed quantity, t the elapsed steps. All sizes are rounded up to whole lots.

Classical schedules.

`immediate` market order for Q at the first decision.

`twap_market` market order for $\lceil R/S \rceil$ every decision.

`twap_passive` each decision: cancel, cross by market order the unfilled part of the previous slice (catch-up), then post the new slice $\lceil (R - \text{catch-up})/S \rceil$ at own best.

`twap_random` slice $\lceil R/S \rceil \cdot U(0.5, 1.5)$; with probability 0.3 a decision is skipped (timing jitter) unless it is the last.

`pov` every decision bring X up to $\min(Q, \lfloor 0.10 \cdot \text{public volume since the start} \rfloor)$; TWAP catch-up over the final tenth of the horizon.

Model-based schedules (parameters from the `Calibration`, §15).

`almgren_chriss_`λ target trajectory $X(t) = Q\,[1 - \sinh(\kappa(T-t))/\sinh(\kappa T)]$, $\kappa = \sqrt{\lambda\sigma^2/\eta}$, for $\lambda \in \{10^{-7}, 10^{-6}, 10^{-5}\}$ ($\kappa T \approx 0.6, 2, 6$ with the default calibration), executed with the passive-with-catch-up tactic: each decision, cross the shortfall against $X(t)$ by market order and post the increment to $X(t+N)$ at own best.

`obizhaeva_wang` with resilience ρ: a block $x_0 = Q/(\rho T + 2)$ at the start and at the end and a constant rate ρx_0 in between, all by market order.

`limit_market` a limit-and-market policy in the spirit of Guéant–Lehalle–Fernandez-Tapia and Cartea–Jaimungal: keep a resting order of two slices at the deepest distance d whose fitted

per-decision fill probability p_d keeps pace with the linear schedule, $p_d \geq \frac{1}{2}(1 - u\,\tau)$ with τ the fraction of time remaining and $u = 0.5$ (quote deeper early, tighten toward the end); keep the order while its price is still the chosen one (queue position); let execution lag the schedule by up to $1/p_d$ slices before market orders catch up; market orders over the last two decisions.

Stealth heuristics.

`footprint` base slice R/S scaled by $U(0.6, 1.4)$; $\times 0.25$ when the executor's own share of the public signed flow exceeds 0.3 (too visible); $\times 1.5$ when the public imbalance exceeds 0.2 in the executor's direction (cover in the natural flow); half of each slice posted passively; TWAP catch-up with market orders over the last 15% of the horizon.

`mimic` draws one of the five institutional execution styles per episode (with the institutional jitter, front-loading and a POV rate $U(0.05, 0.20)$) and runs it, so the executor is statistically one more institution.

15 Calibration (`order_book.execution.calibration`)

The model-based strategies' parameters are fitted under A0 on training seeds by `scripts/calibrate.py`; the defaults in `Calibration` are the values measured on the default market so that every strategy runs without a calibration file.

σ standard deviation of one-step mid changes on the background market (default 0.19 ticks).

η **(temporary impact)** slope of the mean child slippage—fill price minus the mid of the previous step, in the order's direction—on the trading rate Q/T, across TWAP-market runs at $f \in \{0.05, 0.1, 0.2\}$ (default 0.035 ticks per lot/step; the intercept, about 0.55 ticks, is the half-spread).

permanent impact paired impact at completion per lot (default 0.0017 ticks/lot).

ρ **(resilience)** exponential rate fitted by log-linear regression to the paired impact path after completion, relative to its final level (default 0.0028 per step).

depth mean resting lots within 5 ticks of the best on one side (default 227).

fill probabilities a `FillProber` agent posts 1-lot limits at 0, 1, 2 and 3 ticks behind its own best every N steps on alternating sides and cancels them at the next decision; the fill share per distance is p_d (default $(0.28, 0.015, 0.001, 0)$: in a one-tick-spread book only the best is ever reached within 5 steps, which is why passive tactics mostly end up catching up).

16 Held-out and learned detectors

All detectors implement the interface of §9. The A2 and A3 detectors share a `FeatureTracker` that maintains, from the anonymous per-step trade list and the public aggregates: the flow imbalance over 20, 50 and 200 steps; the imbalance over the last 100 lots (volume clock); the signed length of the current run of same-sign aggressive trades; the *regularity* of that run, $1 - \mathrm{CV}$ of its child sizes; the ratio of the trade rate over 20 steps to that over 200; the 50-step return in units of the fast volatility; the spread; and the trade-count imbalance over 20 steps.

A2 (held-out, never used in training).

`runs` $s = \mathrm{sign}(\text{run}) \cdot \mathrm{clip}((\text{run length} - 4)/6, 0, 1)$.

`cusum` two-sided CUSUM with restart on the imbalance x of consecutive 20-step blocks: $S^+ \leftarrow \max(0, S^+ + x - k)$, $S^- \leftarrow \max(0, S^- - x - k)$, $k = 0.1$; when either exceeds $h = 1.5$ an alarm is raised in that direction, both statistics restart, and the alarm is held as a signal decaying linearly to zero over 10 blocks; between alarms $s = (S^+ - S^-)/h$. (A CUSUM without restart stays in alarm because institutions keep the flow one-sided most of the time.)

`volume_clock` the 100-lot imbalance thresholded like the flow detector ($\theta = 0.3$).
`regularity` metaorder reconstruction in miniature (Goliath & Gebbie, 2026): for a same-sign run of at least 3 trades, $s = \text{sign} \cdot \text{regularity} \cdot \min(1, \text{run}/6)$.

A3 (learned). A multinomial logistic classifier on the standardised features with classes {none, buy metaorder active, sell metaorder active} (net side when several overlap), trained with Adam on the class-weighted cross-entropy by `scripts/train_detector.py` on background runs of the A1-medium market (training seeds 100–139, validation seeds 500–507 by default), with labels from the institutional metaorders only. The signal is $p_{\text{buy}} - p_{\text{sell}}$. Training prints one line per epoch (loss, validation ROC AUC for the buy class, the sell class and the unsigned signal) and the fitted weights. The classifier never sees an executor, so its score on the executor measures how institutional the executor looks.

17 Scoring and statistical protocol (`order_book.scoring`)

Detectors. ROC AUC is the rank-based Mann–Whitney statistic with average ranks for ties. In E2 the positives are the steps on which the executor is active and the negatives all other steps of the window (institutional-only and quiet steps alike), so institutional false positives count against the detector exactly as they would in the market. The *signed* AUC uses the signal in the executor's direction. Detection delay is the number of steps from the executor's start to the first signal at or above the operating threshold (0.5) in its direction (the full window when never reached); median and 90th percentile are reported. False-alarm rates are the share of steps with $|s| \geq 0.5$ among steps where only institutions are active and among quiet steps. The same detector's AUC on the institutional metaorders of the episode is reported for comparison.

Statistics. Every comparison is paired: all strategies and adversary levels run on the same seeds, the executor's side is fixed by the seed's parity, and the burn-in state is identical. Means come with 95% percentile-bootstrap intervals over seeds (2 000 resamples); paired differences with a bootstrap interval and a one-sided bootstrap p-value for "> 0"; Holm's step-down correction is applied over the family of leakage tests; tail risk is CVaR at 95%; results are also reported by side. The per-episode shortfall is dominated by the market's own drift over the horizon (standard deviation of order 10 bps on a 2 000-step horizon), which the pairing removes: in a four-seed pilot the paired leakage interval was already ± 0.4 bps.

18 Experiments

18.1 Adversary levels and observers

Level	Market	Role in Phase 2
A0	four makers, none reacts; no anticipator	impact without leakage; calibration
A1-{lenient, medium, strict}	two of four makers react to the flow detector at $\theta_d = 0.2/0.3/0.45$	the training-time adversary; E3 uses medium
A4 Observers	A1-medium plus one order anticipator flow at three thresholds (A1), runs, CUSUM, volume clock, regularity (A2), learned (A3)	the aggressive adversary of E3 attached in E2; they see the market, nobody reacts to them

Observers are separate detector instances registered under `obs:<name>`, so the detector the makers react to (`flow`) is also recorded. Seeds 0–999 are training seeds (calibration, detector

training, tuning); test seeds start at 1000. The executor buys on even seeds and sells on odd seeds.

18.2 E2: detectability of classic strategies

`E2Config`: strategies {twap_market, twap_passive, twap_random, pov, almgren_chriss_1e-06, footprint, mimic} × sizes {0.05, 0.10, 0.20} × 20 test seeds, in the A1-medium market with every observer attached; $T = 2\,000$, $N = 5$, burn-in 2 000, 500 steps recorded after the horizon. Per strategy, size and detector: AUC of $|s|$ with bootstrap interval, signed AUC, delay (median, p90), false alarms on institutional-only and quiet steps, and the detector's AUC on institutional metaorders. The per-step signals of every episode are saved (`outputs/e2/episodes.jsonl`) so ROC curves can be drawn without re-simulating.

18.3 E3: leakage cost

`E3Config`: all twelve baselines × sizes {0.05, 0.10, 0.20} × adversaries {A0, A1-medium, A4} × {visible, invisible} × 20 test seeds, plus one run per seed and adversary with no executor (the market's own path). "Invisible" means `Market.hide` is applied to the executor: detectors and the public flow statistics ignore its trades while everything else is unchanged, so on the same seed

$$\mathrm{IS}_{\mathrm{visible}} = \underbrace{\mathrm{IS}_{\mathrm{invisible}}}_{\text{impact cost}} + \underbrace{\mathrm{IS}_{\mathrm{visible}} - \mathrm{IS}_{\mathrm{invisible}}}_{\text{leakage cost}}. \tag{19}$$

Under A0 the two runs are identical to the last digit (nobody reads the detector), which the tests assert; under A1 and A4 the difference is caused by detection alone up to the state dependence of §7. Per strategy, size and adversary the summary reports n, mean IS visible and invisible with intervals, the paired leakage with interval, p-value and Holm-adjusted p-value, CVaR_{95}, completion, forced share and results by side. With `-paths` the mid paths are saved and the impact path (executor minus no-executor run) is written to `outputs/e3/impact_paths.npz`. The script ends with the Phase 2 gate verdict: TWAP (market) leakage > 0 with $p < 0.05$ at one size or more under A1 or A4.

Pilot. A four-seed pilot of TWAP (market) at the full horizon gave a paired leakage of +0.37 bps [0.08, 0.83] at $f = 0.10$ under A1-medium ($p < 0.001$), −0.25 under A4 at the same size, and −0.01 / +0.17 at $f = 0.20$: the gate passed in the pilot, but the effect is small (tenths of a basis point against an impact cost of a few basis points) because the flow detector is rarely confident at these participation rates ($|s| > 0.5$ on 1–7% of steps). The full E3 run decides; per the plan, if leakage is not significant the result is reported before detector strength or cover is recalibrated.

19 Reproduction of Part II

```
# 1. calibration of the model-based baselines under A0 (training seeds 0-9; ~3 min)
python -m scripts.calibrate
# 2. the A3 classifier (40 training + 8 validation seeds x 20k steps, 30 epochs,
    verbose per epoch)
python -m scripts.train_detector
# 3. E2: detectability (7 strategies x 3 sizes x 20 seeds = 420 episodes; ~2 min with
     8 workers)
python -m scripts.e2_detectability --workers 8
# 4. E3: leakage (12 strategies x 3 sizes x 3 adversaries x 2 x 20 seeds + 60 = 4,380
     episodes;
#    ~15-20 min with 8 workers); --paths also saves impact paths
python -m scripts.e3_leakage --workers 8 --paths
```

```
# smoke tests of every script (seconds):
python -m scripts.calibrate --quick; python -m scripts.train_detector --quick
python -m scripts.e2_detectability --quick; python -m scripts.e3_leakage --quick
```

Each episode takes about 0.3 s on one core (burn-in 2 000 + 2 500 recorded steps); every script prints one line per episode with seed, strategy, size, adversary, visibility, side, IS, executed and forced quantities, elapsed time and ETA, and saves JSON lines of every episode plus a summary JSON.

Part III
Verification and status

20 Tests

`python -m pytest` runs the suite (115 tests) in about five seconds:

`test_book.py` order book rules: empty and one-sided states, exact window boundaries and reported evictions, FIFO and partial fills on both sides, price protection of crossing limits, all four self-trade modes after an external partial fill, cancel/modify priority, atomic rejection of invalid modifications, input validation including NumPy integers, snapshot/array isolation, overflow of aggregate volume, and a 600-action random sequence with the full invariant check after every action.

`test_market.py` reservation bookkeeping under every removal path (fill, cancel, STP, eviction, budget cap) and the public flow window.

`test_simulator.py` determinism, conservation, snapshot/restore and reseed, analysis-mode logging, population built from the configuration with a shared detector registry, frozen and replaceable configuration, flow detector thresholding and saturation.

`test_foundations.py` exact event-log replay from (config, seed), from a snapshot and after reseed; stream independence from population changes; invariants after every step; the no-hidden-information guard (with an anticipator present and the metaorder labels guarded), which also verifies that it catches a leak.

`test_agents.py` every execution style completes on both sides, is labelled and leaves no resting order; metaorders arrive with log-normal sizes, balanced sides and participation in range; the side choice reverts inventory; the anticipator follows the signal within its limit and unwinds; the contrarian anticipator trades against the flow first; makers pull quotes on a volatility spike and return after the cooldown, and reduce size with relative volatility; private valuations run and differ from the shared value; an idle external agent leaves the event log and every stream unchanged, and a trading one leaves the scheduler and hidden-value streams unchanged; adversary presets.

`test_env.py` the Gymnasium prototype is seeded, bounded and picklable.

`test_execution.py` every baseline completes Q on both sides within the horizon (plus grace), leaves no resting order, reports a finite IS and consistent fills; IS equals the tape-derived average price; Immediate sends one order and TWAP one per decision; a do-nothing strategy is forced in full at T; the resting-order, through-ticks and participation limits bind; `Market.hide` removes the executor from the flow window and trade list but not from volume or the tape; Almgren–Chriss trajectories are monotone, front-loaded for higher λ and end at Q; Obizhaeva–Wang blocks and rate sum to Q; the resilience fit recovers a known decay; calibration files round-trip; the fill prober works.

`test_detectors.py` feature tracker runs, regularity and shapes; the A2 detectors follow one-sided flow and flip; the CUSUM alarms, restarts and decays; the A3 classifier trains to AUC > 0.9 on separable synthetic data, round-trips its weights and signs its signal correctly;

observers attach to a simulator and update every step (A3 skipped without weights).

`test_scoring.py` AUC on perfect, inverted, tied, random and degenerate inputs; detection delay and false alarms; signed activity labels; bootstrap intervals and paired differences; CVaR; Holm.

`test_experiments.py` episode sides by parity and spec counts; an episode is deterministic and JSON-serialisable; visible and invisible runs are identical under A0 and share Q and arrival mid under A1; the E3 runner writes one record per episode, summarises with leakage, Holm and gate fields and yields one impact path per visible episode; the E2 summary covers every observer with valid scores.

The hidden-information guard of `test_foundations.py` now also wraps an executor running the `footprint` strategy (the baseline that reads the most state).

21 Reproduction

```
pip install -e .                                    # or: uv pip install -e .
python -m pytest                                    # test suite
python -m scripts.benchmark                         # steps/s and a profile
python -m scripts.validate                          # 5 seeds x 10k steps, with paired
    impact
python -m scripts.validate --seeds 10 --steps 50000 --preset private
python -m scripts.impact --seeds 20 --styles twap_market twap_passive pov
python -m scripts.run                               # analysis run -> outputs/
    simulation.png
python -m scripts.batch                             # 100 seeds in parallel -> outputs
    /price_paths.*
python -m scripts.animate                           # video with the live book ->
    outputs/simulation.mp4
python -m scripts.burn_in --seeds 8                 # RL start-state bank -> outputs/
    states.pkl
cd docs && latexmk -pdf simulator.tex               # this document
```

Every script writes to `outputs/` (ignored by git) and takes `-seeds`/`-steps` arguments; the defaults finish in seconds, the paper runs in minutes. `validate` and `impact` save JSON (and the impact paths as `.npz`) so that figures can be rebuilt without re-simulating.

22 Status against the research plan and next steps

Phase 0 (foundations). Complete: invariant tests, deterministic replay, benchmark, snapshot/restore, per-agent streams, no-hidden-information guard.

Phase 1 (simulator). All items of §4.1 of the plan are implemented: the fundamental switch, the institutional population with labels, detectors behind one interface, configurable maker responses and fragility, the order anticipator, the step unit and training support. Every §4.2 target passes on 5 seeds × 30 000 steps with the final defaults (Table 3), so the Phase 1 gate is met for the default (shared-value) market. Two caveats for the paper run: the paired impact exponent rests on $n = 25$ injected orders and should be re-estimated with `python -m scripts.impact -seeds 20` (and across styles); and the private-valuation variant needs its own calibration before results are reported under it. The optional comparison with a LOBSTER sample using LOB-Bench metrics has not been done.

Phase 2 (measurement without RL). Implemented (Part II): the execution task and executor, twelve baselines with calibration, detectors A2 and A3, scoring and the statistical

protocol, and the E2/E3 definitions, runner and summaries, all covered by tests. **The gate has not been run at full scale**; the four-seed pilot passed it narrowly (§18.3). The commands of §19 produce the E2 and E3 tables; the gate verdict is printed at the end of E3. Open items: the Goliath–Gebbie reconstruction is represented by the simplified regularity detector; the A3 classifier is logistic rather than a GRU or gradient boosting (both can be added behind the same `LearnedDetector` interface); the Gymnasium prototype (`env.py`) is still the pre-plan environment and will be replaced by the execution MDP of plan §5 in Phase 3, reusing `ExecutorAgent` and `ExecState`.